Introduction

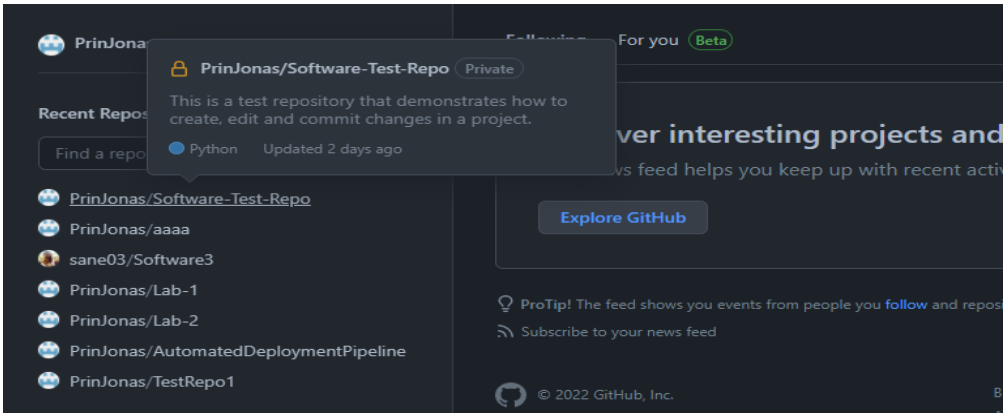
These notes are a continuation of introduction to GitHub. Git branching, Merging and pull requests is explored. To understand these notes it is imperative you should have studied the preceding notes on GitHub repositories and working with repositories.

Git Branching, Merging and Pull Requests

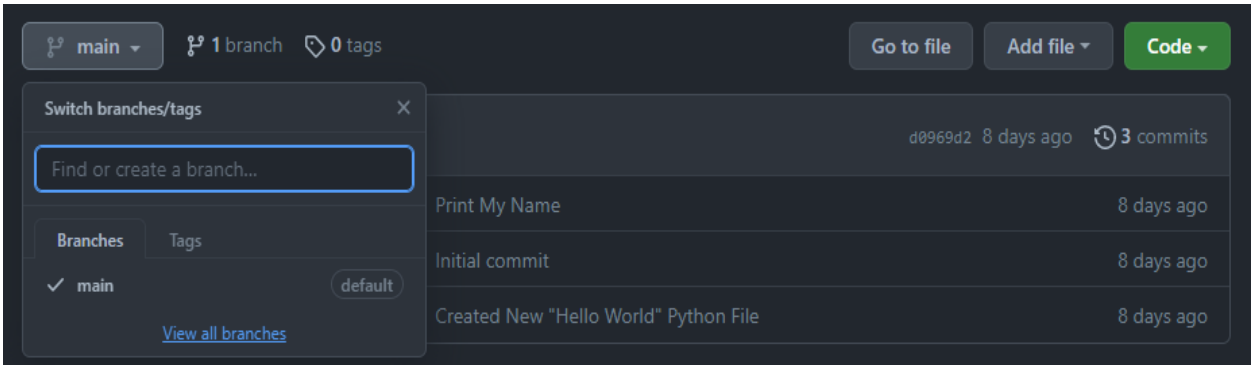
After having created a repository, all the commits done thus far have been on the default branch of the project. In terms of version control, a branch is simply defined as a version (trail of valid commits) of the project at any particular time. For instance, all the changes made until now form the main branch version of the repository. It follows that users in a development team can create new separate branches when developing new large features of a project. Any new branch created is initialized (contains all the progress and source code) from other existing branches in the repository. This is advantageous when collaborating; new functionality can be added onto a project without affecting the main branch. For example, if a social media platform wanted to add a new feature as part of an update, the team would create a separate feature branch where the new idea can be explored. This new feature branch does not affect the main branch until such a time when the changes are deemed valuable to the project. Finally, feature branches can be merged back on to the main branch, and the idea of raising pull requests before merging is essential for the team to review the changes and safely merge them without errors.

Branching

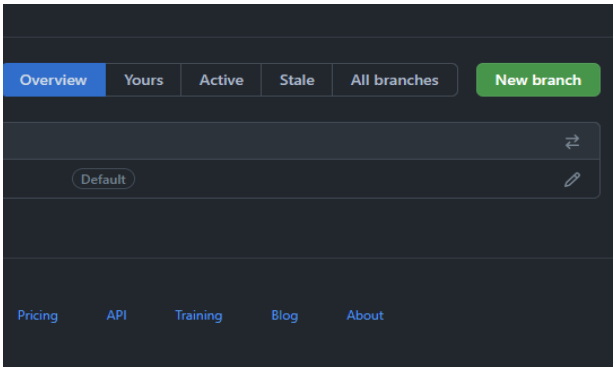
As previously stated, whenever a new repository is created, a default branch named main is created to store the version of the project. By navigating to your GitHub home page, click on the repository as shown in the illustration below that you created in the previous section of this guide.



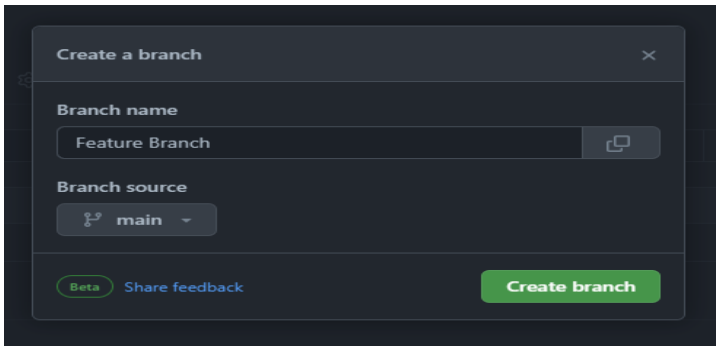
To create a new branch in your repository, proceed to click on the drop down button labeled main which should show you a list of all the branches in your project. For now, there is just the main branch, select the View all branches option to see a comprehensive list of all the branches. This is shown in the illustration provided below.



Note that version control also keeps track of how often a repository is updated. Click on the New branch button on the top-left of your screen as shown below.



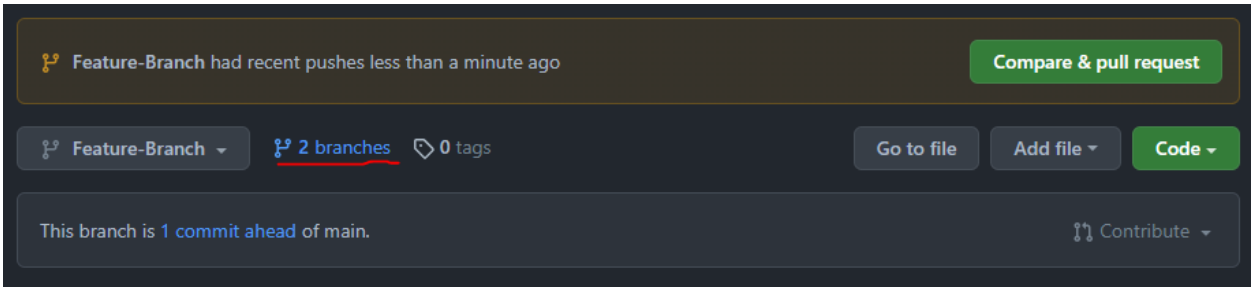
This should produce a dialogue box which prompts you about the specific details of the branch you want to create. You need to provide a valid branch name, of most importance is that there is an option to specify the existing branch from which the new branch will be created. In essence, this implies that the new branch will have the progress that you have made thus far on the main branch. A new branch creates a copy of the existing project such that when new changes are made and are found to be faulty, the original copy of your project still exists. Now, different team members can have copies of the same source code, while being able to make their changes independently. Follow the prompts and click on Create branch as shown in the illustration below.



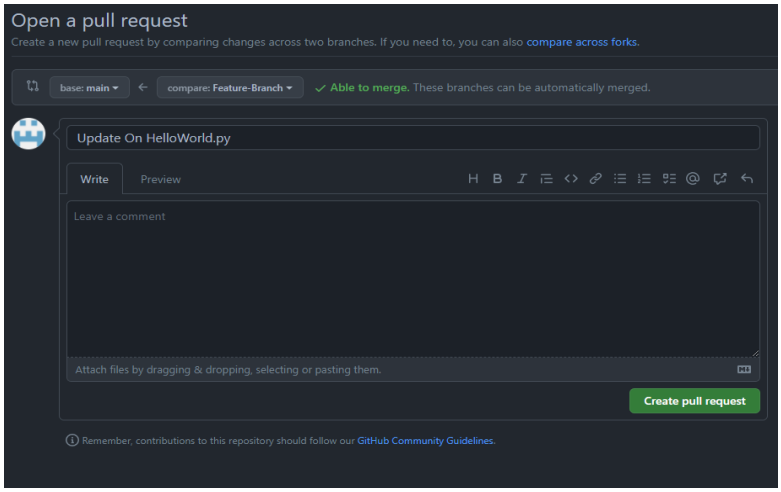
Now that you have created a new branch, create more commits and add more changes to your project.

Merging Branches and Raising Pull Requests

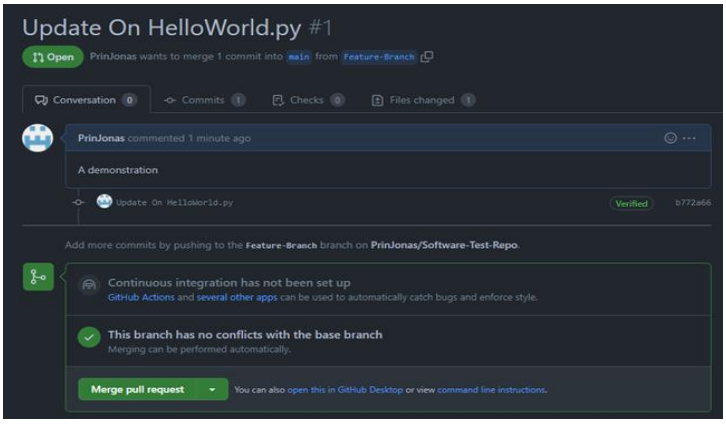
After having made some changes to your new branch, this new copy of the project can be merged back onto the main branch. Simply, merging is the process of comparing the new branches to the main branch and subsequently combining the two if there are no errors or conflict. A common error when attempting to merge is that known as merge conflicts. Merge conflict occur when the same file is changed or edited on both branches and at the same places. For instance, if you and your project partner create new branches from the same main, but you both apply changes on the same lines of source code. GitHub faces some conflict or confusions for lack of better words as to which changes to keep. A good practice to avoid these merge conflict is to raise what are known as pull requests. To illustrate this, click on the branches option as shown in the illustration below.



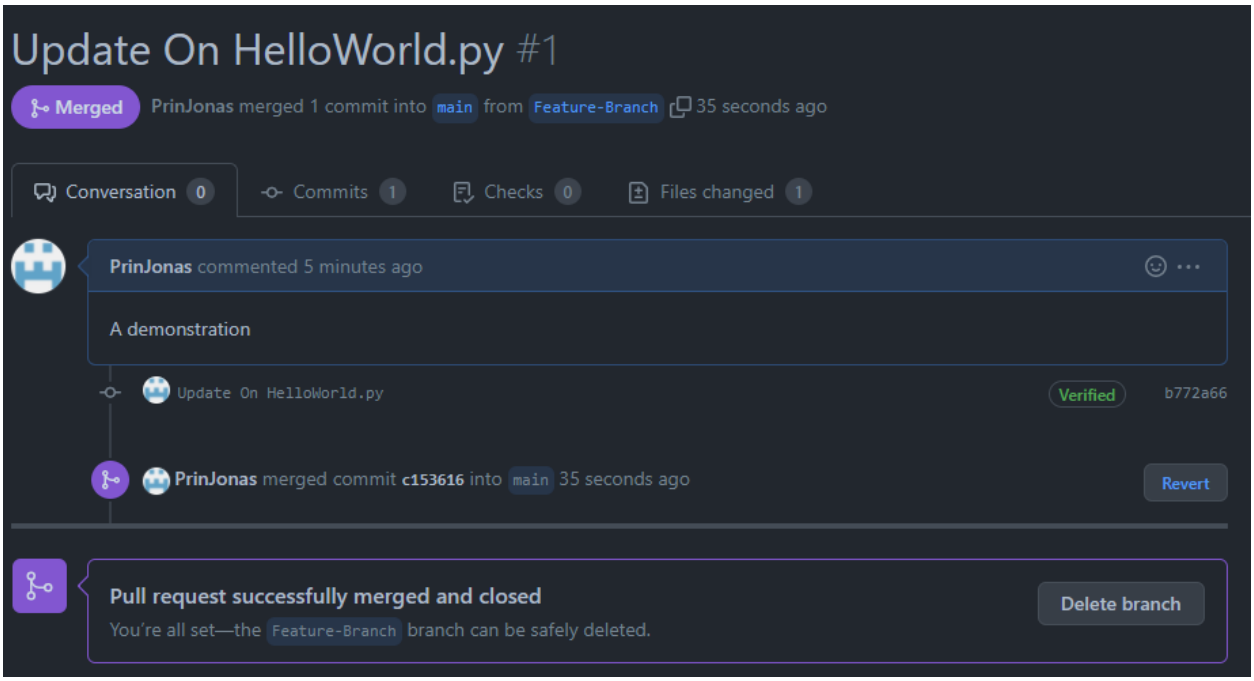
GitHub already picks up on the differences between the two branches. On the subsequent page, select the New pull request option also as depicted in the illustration below. This will lead to a page prompting about the specific details of the pull request you are raising.



In this example, all the changes made are not in conflict with the main branch and thus a display message is shown that the branch can be merged back. When a pull request is raised, the developer needs to provide a name and a brief description of what changes they are requesting to be pulled by the other members. Continue and provide the necessary details then click on Create pull request. The illustration below shows that you now have one pull request under the Pull Requests tab of your repository. Other team members in your group may review these changes in a safe way and either approve or reject the changes made.



Note that a new pull request will retain its open status until members of the repository approve that the changes can be merged. If there are no conflicts as with this example, proceed to merge the pull request and confirm this action. The illustration below shows that the status of the pull request has changed from open to merged.



If you check your main branch once more, you will see that the changes made on the new feature branch have been integrated into the main branch of the project. This is such a powerful too because now software development teams can collaborate and work remotely on the same projects in an independent fashion. The process of raising and reviewing pull requests allows the team to make collective decisions on new features and how to adopt efficient workflows.